

MDAO Ecosystem

Independent Smart-Contract Security Review

Auditor soldrift (GitHub)

Report v2.0 · 2026-04-26 / 27 (UTC+8)

Scope MleeDao.sol, MDAOPartnership

Token (BSC) 0xC4bF2518AA953304170753388F83277Eb9588a7A

MDAO Smart Contract Security Assessment

Document version: 2.0

Assessment date: 2026-04-26 (UTC+8)

Last updated: 2026-04-27 (UTC+8)

Field	Detail
Lead assessor	Ming Cheng, operating publicly as soldrift — GitHub
Publication	github.com/soldrift/mdao-audit-report
Scope	Mleedao.sol, MDAOPartnership (Solidity sources; SHA-256 in § Cryptographic attestation)
Network	BNB Smart Chain (BSC)
Primary token (BSC)	0xC4bF2518AA953304170753388F83277Eb9588a7A
Engagement type	Independent, fixed-scope, read-only source review

Assessor identification

Item	Detail
GitHub	https://github.com/soldrift
X	https://x.com/hodlwarden
Prior affiliation	Former association with Cyberscope (blockchain security). This assessment is not issued by Cyberscope and does not imply Cyberscope's review, approval, or endorsement.

Citation (third parties):

"Independent security assessment by Ming Cheng ([soldrift](#)). Materials are published at [soldrift/mdao-audit-report](#), including the cryptographic attestation at the end of this document."

Stakeholders should rely on a **specific git commit** for which the § **Cryptographic attestation** values and the cited Solidity digests remain unchanged.

Executive summary

Within the stated scope and methodology, **no critical- or high-severity vulnerabilities** were identified in the reviewed source code. **No permissionless attack path was identified** that would allow a third party (without owner privileges or compromised user keys) to misappropriate other users' token balances, bypass required payments where applicable, double-claim vesting entitlements, or violate the on-chain solvency accounting for outstanding allocations.

The **MLEEDAO** token, as reviewed, follows OpenZeppelin **ERC-20** and **EIP-2612 permit** patterns with **fixed supply** minted at deployment; the reviewed implementation does not introduce mint, transfer-tax, pause, or proxy mechanisms that would create an unauthorised third-party drain surface in the token contract itself.

The **MDAOPartnership** contract, as reviewed, applies **reentrancy protection** on sensitive entrypoints, **ordering consistent with checks-effects-interactions** on the paths examined, **solvency checks** before increasing aggregate

obligations, and **burn logic that does not consume MDAO reserved for existing vesting claims**.

Residual items are predominantly **governance, administrative configuration, and operational custody** (e.g. owner-controlled parameters, treasury behaviour), not demonstrated permissionless exploits in contract logic.

Scope of work

In scope: Source files `Mleedao.sol` and `MDAOPartnership.sol` (SHA-256 in § Cryptographic attestation); alignment with remediations described in the project's partnership audit-fix documentation where applicable.

Out of scope: Deployment configuration, off-chain systems, key management, economic or legal advice, formal verification, exhaustive dynamic testing, and any contracts or versions not included in the checksum list.

Methodology

Static manual review: control-flow and state-transition analysis, reentrancy and external-call ordering, economic and access-control invariants, and comparison of implementation to documented intent. **No on-chain execution, fuzzing, or formal proof** was performed as part of this engagement.

System description (summary)

Token (MLEEDAO): Constructor mints `18_000_000_000 * 10^decimals()` to the deployer; standard ERC20 + ERC20Permit.

Partnership (MDAOPartnership): Whitelist-gated onboarding; stablecoin payments with on-chain fee distribution; optional MDAO burn to a null address subject to balance constraints; tiered lock and linear vesting; administrative parameters with timelocking where implemented for certain changes.

Adversarial analysis (“can we break it?”)

This section answers whether an attacker who is **not** the owner could steal funds or corrupt accounting through public or whitelisted flows.

MLEEDAO (token)

Attack idea	Outcome
Mint more supply after deploy	Not possible — no <code>mint</code> ; constructor runs once.
Steal balances via non-standard <code>transfer</code>	Not possible under normal ERC-20 assumptions; OpenZeppelin ERC20 is used.
Drain others via <code>permit</code> + <code>transferFrom</code>	Needs victim signature — phishing / malicious dApps; not a contract logic hole in isolation.

MDAOPartnership (non-owner attacker)

Attack idea	Outcome
Re-enter <code>claimVested</code> / onboarding and double-spend	Mitigated — <code>nonReentrant</code> on critical paths; effects before external token calls in the main flows reviewed.
Onboard twice (company + individual + partner)	Blocked — mutual-exclusion flags.

Force onboarding without paying (when payment required)	Blocked — <code>safeTransferFrom</code> must succeed; paid amount derived from recorded <code>tokenPrice</code> and <code>_mdaoAmount</code> .
Drain MDAO owed to others while keeping your allocation	Blocked — <code>withdrawToken</code> on MDAO cannot reduce balance below <code>totalMdaoAllocated</code> - <code>totalMdaoClaimed</code> ; new obligations require solvency pre-check vs balance.
Burn path steals vesting reserves	Mitigated — burn only if <code>mdaoBurnAmt</code> + <code>lockedObligation</code> <= balance; otherwise <code>BurnSkipped</code> .
Steal fee wallets' past proceeds	N/A to contract — stablecoins already forwarded to configured wallets in the same transaction.

Where a “break” is still realistic (trust / governance)

These are not typical “hack the bytecode” issues; they assume **malicious, negligent, or compromised owner keys**:

Scenario	Effect
Owner sets extreme <code>burnBps</code> in same block as your onboard	Does not steal <i>your</i> allocation line-by-line, but drains protocol MDAO toward burn; economic / fairness concern.
Owner withdraws all free MDAO (above reserves)	New onboards / partners revert until refill; can grief future participants; existing claimants remain protected by reserve math.
Owner pauses indefinitely	Mitigated — anyone can <code>forceUnpause</code> after 30 days.
Owner never whitelists you	Griefing — you cannot onboard; not direct fund theft from others.

Bottom line: Under the reviewed code, a **random on-chain attacker** who is not the owner and cannot forge signatures is **not shown a path to steal other users' vested MDAO or bypass payment rules** — i.e. the **contract is not compromisable** by that class of attacker in the flows examined. The main residual “break” story is **trusted admin behaviour** (key compromise, aggressive parameters, operational constraints on *future* capacity), which is **key custody / governance**, not a permissionless code exploit.

Findings — `MleeDao.sol`

ID	Severity	Issue	Recommendation
MLEE-01	Medium	Full initial supply minted to deployer	Treasury multisig, vesting, and published distribution
MLEE-02	Info	EIP-712 domain tied to name/version/chain/address	Acceptable for v1; adjust on redeploy if needed
MLEE-03	Info	<code>decimals()</code> not restated in child contract	Optional NatSpec
MLEE-04	Info	Long-lived permit deadlines (wallet / UX)	End-user guidance

Findings — MDAOPartnership

ID	Severity	Issue	Recommendation
MDP-01	Medium	setBurnBps effective without timelock	Align with timelocked admin changes
MDP-02	Medium	Owner may withdraw MDAO above reserved claims	Timelock / multisig; optional transparency helpers
MDP-03	Low	Multiple fee <code>safeTransfer</code> calls	Acceptable for standard stablecoins; <code>nonReentrant</code> on onboard
MDP-04	Low	Price event at cap	Optional single cap-reached event
MDP-05	Low	Pause / force-unpause cadence	Optional cooldown
MDP-06	Low	Large <code>partners</code> batch gas	Per-call cap
MDP-07	Info	<code>mdaoToken.decimals()</code> not sanity-checked	Mirror payment-token checks
MDP-08	Info	Burn skip vs fee split semantics	Disclosure
MDP-09–10	Gas	Minor optimisations	Optional

Administrative capabilities

The owner may pause (subject to permissionless unpause after a delay), manage whitelist, adjust burn basis points, withdraw tokens subject to solvency rules, deposit MDAO, allocate partner rewards within caps, and execute timelocked updates for certain parameters. **Production use of a multisig and operational discipline is strongly recommended.**

Cryptographic attestation (EIP-191)

The assessor binds the **exact UTF-8 bytes** of the **canonical message** below (Solidity file digests, the SHA-256 of all narrative in this file strictly before this section, the BSC token address, and a dated statement) to an **Ethereum** key using **EIP-191** (`personal_sign`). The signature is **off-chain** (no transaction, no gas). **Any** change to the narrative above this section, to the canonical message, or to the cited sources invalidates the attestation until re-signed.

Off-chain signature: run `python docs/sign_attestation_eth.py` (reads `AUDITOR_PRIVATE_KEY` from `.env`; see `.env.example`) and refresh the values below if the canonical message changes.

Field	Value
Signatory	Ming Cheng (soldrift)
Signer address	<code>0x19E7E376E7C213B7E7e7e46cc70A5dD086DAff2A</code>

Scheme	EIP-191 (personal_sign) over the canonical message (UTF-8)
---------------	--

Canonical message (UTF-8, exact bytes signed):

```
I, soldrift (legal name: Ming Cheng; https://github.com/soldrift; https://x.com/hodlwarden), confirm that I performed an independent security review of the following Solidity sources: - MleeDao.sol sha256=dad6bfff992685e12cac23df30df78c60d210d4b2d08b32bcc375212757161838 - MDAOPartnership.sol sha256=4bc58c1797cc16e9d854ab7fbcc7f0d8c964ef80bbc8b1a3f603d29caa25fca1 (on-chain contract name: MDAOPartnership) The assessment narrative in AUDIT-REPORT-soldrift.md (every byte strictly before the heading "Cryptographic attestation (EIP-191)") has SHA-256: b51f219c1581fca17ac280bafb0a07ae4930bb8857a914cf6364f4955d76a200 This message is dated 2026-04-27 (UTC+8) and binds only the exact bytes above. Any change to those files invalidates this attestation. Primary contract under review (BSC): 0xC4bF2518AA953304170753388F83277Eb9588a7A
```

SHA-256 of the canonical message (hex):

1962e29d272810e1b3981a780b5ebe58562b2c578a2fd303108cc6ee5fa1f1da

ECDSA signature (hex, 65 bytes, EIP-191) — one logical 0x... string; line break after r (32 bytes) for readability:

```
0x5156dc081a699256912d3290dcb1992b2a08dfc12dbd5a9fe71cddf75afad147
32226e230634582d504a7500922e970b00bfc6ee3da661cf5b763b1b1bdfed91b
```

End of document